

INFORME TECNICO

Grupo Cordillera

Arquitectura C1, C2, C3; patrones; modelo de datos; decisiones criticas

Campo	Detalle
Sistema	Dashboard empresarial Grupo Cordillera
Stack principal	React 19.2.5, TypeScript 6.0.2, Vite 8.0.10, KrakenD 2.13, Node.js 22, NestJS 11.1.9, Java 25, Spring Boot 4.0.6, PostgreSQL 16
Alcance	Frontend, API Gateway, BFF, microservicios, persistencia y endpoints publicados
Lectura objetivo	Maximo 10 minutos

Resumen ejecutivo

El sistema separa presentacion, entrada de API, adaptacion para frontend, dominios backend y persistencia. El frontend consume solo endpoints publicados por KrakenD; el BFF enruta esas llamadas; los microservicios Spring Boot resuelven autenticacion, usuarios, KPIs y reportes. La vista de reportes ya lista y crea reportes usando GET y POST /api/reports.

1. Arquitectura C1 - Contexto

Grupo Cordillera entrega un panel para login, creacion de usuarios, dashboard de KPIs, alertas y gestion basica de reportes. El usuario interactua con React; no accede directamente a microservicios ni bases de datos.

Usuario de negocio

- > Frontend React/TypeScript
- > API Gateway KrakenD
- > BFF NestJS
- > Microservicios Spring Boot
- > PostgreSQL por dominio

Actor/Sistema	Responsabilidad
Usuario	Opera login, KPIs, alertas, creacion de usuarios y reportes.
Frontend	Renderiza pantallas y consume contratos HTTP publicos.
API Gateway	Publica solo endpoints usados por la UI y delega al BFF.
BFF	Actua como fachada/proxy hacia microservicios.
Microservicios	Implementan capacidades por dominio.

Actor/Sistema	Responsabilidad
PostgreSQL	Persiste usuarios, KPIs y reportes en bases separadas.

2. Arquitectura C2 - Contenedores

La solución se ejecuta con contenedores independientes. En Docker, el frontend sirve React con Nginx 1.27 y reenvía /api al gateway. KrakenD escucha en 8088 y delega al BFF en 8000. Los servicios Java usan PostgreSQL 16 por dominio.

Contenedor	Version/tecnología	Rol
front-web2	Node.js 22 build, React 19.2.5, Vite 8.0.10, Nginx 1.27	UI y proxy local /api hacia gateway.
api-gateway	KrakenD 2.13	Entrada pública controlada para /api.
bff-service	Node.js 22, NestJS 11.1.9, TypeScript 5.9.3	Proxy orientado al frontend.
auth-service	Java 25, Spring Boot 4.0.6, Nimbus JOSE JWT 10.5	Login y JWT RS256.
user-service	Java 25, Spring Boot 4.0.6, JPA, Flyway	Usuarios y credenciales.
kpis-service	Java 25, Spring Boot 4.0.6, JDBC, Flyway	Indicadores, gráficos y alertas.
report-service	Java 25, Spring Boot 4.0.6, JPA, Flyway	Listado y creación de reportes.
PostgreSQL	PostgreSQL 16	Bases user_db, kpis_db y report_db.

```

Browser -> front-web2
/api/* -> api-gateway:8088
-> bff-service:8000
-> auth-service | user-service | kpis-service | report-service
-> user_db | kpis_db | report_db

```

3. Arquitectura C3 - Componentes

Contenedor	Componentes	Funcionamiento
BFF	AuthController, UsersController, KpisController, ReportsController, clientes HTTP, JwtGuard	Expone solo rutas usadas por UI: login, crear usuario, KPIs y GET/POST reportes.
ms-auth	UserController, UserService, JwtService, HttpClient	Autentica delegando a ms-user y firma JWT RS256.
ms-user	UserController, UserManagementService, UserRepository, User	Valida, autentica y persiste usuarios.
ms-kpis	KpiController, KpiQueryService,	Selecciona estrategia por KpiType y consulta tablas KPI.

Contenedor	Componentes	Funcionamiento
	KpiStrategyFactory, estrategias, repositorio JDBC	
ms-report	ReportController, ReportService, ReportRepository, Report	Valida, lista y crea reportes.

4. Endpoints publicados y usados por el frontend

Metodo	Endpoint publico	Pantalla/uso	Destino final
POST	/api/auth/login	Login	ms-auth -> ms-user
POST	/api/users	Crear usuario	ms-user
GET	/api/kpis/summary	Dashboard/KPIs	ms-kpis
GET	/api/kpis/sales/monthly	Dashboard	ms-kpis
GET	/api/kpis/branches/performance	Dashboard	ms-kpis
GET	/api/kpis/channels	Dashboard	ms-kpis
GET	/api/kpis/alerts	Dashboard/Alertas	ms-kpis
GET	/api/reports	Listar reportes	ms-report
POST	/api/reports	Crear reportes	ms-report

Contrato publico reducido

KrakenD no funciona como wildcard abierto. Endpoints como /api/dashboard, /api/auth/register, /api/auth/public-key, /api/users/authenticate, /api/kpis/{type} y /api/reports/{id} no se publican por gateway en el escenario actual; solo quedan para uso directo de microservicio, pruebas Swagger o integracion interna.

5. Patrones de arquitectura y disenio

Patron	Donde aparece	Proposito
API Gateway	Backend/api-gateway/krakend.json	Entrada unica y contrato explicito de endpoints publicos.
Backend for Frontend	Backend/bff	Fachada/proxy adaptada al frontend.
Microservicios	ms-auth, ms-user, ms-kpis, ms-report	Separacion por dominio y despliegue independiente.
Layered Architecture	Servicios Java	Controller -> Service -> Repository -> Database.

Patron	Donde aparece	Proposito
Repository	UserRepository, ReportRepository, repositorio KPI	Encapsula acceso a datos.
DTO	Paquetes dto e interfaces TypeScript	Separa contratos HTTP de entidades internas.
Strategy	Estrategias KPI y estrategias visuales React	Permite variar logica por tipo de KPI o estado.
Factory	KpiStrategyFactory	Resuelve estrategia por KpiType.
Dependency Injection	Spring y NestJS	Reduce acoplamiento y facilita pruebas.
Client Adapter	UserClient/HttpUserClient en ms-auth	Desacopla auth del proveedor de usuarios.

6. Modelo de datos

El proyecto mantiene base por servicio. Flyway versiona esquemas; JPA valida esquema en user/report y JdbcTemplate consulta datos KPI.

Base	Tabla	Campos clave	Servicio
user_db	users	id, username, email, password, first_name, last_name, role, created_at	ms-user
kpis_db	kpi_summary	ventas_totales, margen_utilidad, stock_critico, reclamos_activos, ticket_promedio, satisfaccion_cliente	ms-kpis
kpis_db	monthly_sales	month_label, sales_value	ms-kpis
kpis_db	branch_performance	branch_name, score	ms-kpis
kpis_db	sales_channels	channel_name, percentage	ms-kpis
kpis_db	alerts	id, title, status, date_label, description	ms-kpis
report_db	reports	id, title, description, report_type, status, generated_at	ms-report

7. Decisiones criticas

1. Usar KrakenD 2.13 como gateway con endpoints declarados explicitamente para evitar exposicion innecesaria.
2. Mantener BFF NestJS 11.1.9 para desacoplar el frontend de URLs internas y centralizar proxy.
3. Separar dominio en microservicios Java 25/Spring Boot 4.0.6 para autenticacion, usuarios, KPIs y reportes.
4. Usar PostgreSQL 16 con base por servicio: user_db, kpis_db y report_db.

5. Versionar esquemas con Flyway para despliegues repetibles.
6. Usar Strategy/Factory en KPIs para extender indicadores sin reescribir el servicio principal.
7. Publicar GET y POST /api/reports porque la UI actual ya lista y crea reportes desde el front.

8. Riesgos y mejoras recomendadas

- El BFF valida solo formato Bearer, no firma JWT; conviene validar contra la llave publica de ms-auth.
- Las passwords seed estan en texto plano; para produccion se debe usar hashing como BCrypt.
- ms-auth conserva endpoints directos de register/users/public-key para pruebas, pero no se publican por gateway.
- Cada endpoint nuevo del frontend debe declararse en KrakenD y en el BFF; no hay proxy wildcard publico.
- El bundle del frontend supera 500 kB; podria dividirse con lazy loading si se busca optimizacion.

9. Conclusion

El proyecto queda alineado con una arquitectura distribuida y controlada: React consume un contrato minimo por KrakenD, el BFF enruta, los microservicios concentran reglas por dominio y PostgreSQL persiste por servicio. La documentacion y el informe reflejan el estado actual: reportes se pueden listar y crear desde el frontend; endpoints no usados permanecen fuera del gateway.

10. Pruebas en Swagger

Swagger UI se prueba directamente en cada microservicio Java, no desde KrakenD. Para ejecutar las pruebas, levantar el stack con docker compose up --build, abrir la URL Swagger del servicio, seleccionar el endpoint, usar Try it out, completar parametros o body y presionar Execute.

Servicio	Swagger UI	OpenAPI JSON
ms-auth	http://localhost:9080/swagger-ui/index.html	http://localhost:9080/v3/api-docs
ms-kpis	http://localhost:9081/swagger-ui/index.html	http://localhost:9081/v3/api-docs
ms-user	http://localhost:9082/swagger-ui/index.html	http://localhost:9082/v3/api-docs
ms-report	http://localhost:9083/swagger-ui/index.html	http://localhost:9083/v3/api-docs

10.1 Auth Service

Metodo	Endpoint	Prueba Swagger	Resultado esperado
GET	/api/auth/health	Ejecutar sin body.	{"status":"UP","service":"auth-service"}
POST	/api/auth/login	Body: {"username":"vendedor","password":"1234	Respuesta 200 con username, role, tokenType Bearer, expiresIn y accessToken.

Metodo	Endpoint	Prueba Swagger	Resultado esperado
		"}	
POST	/api/auth/register	Body con email, username, password, firstName, lastName y role.	Respuesta 201 con mensaje de usuario registrado.
GET	/api/auth/users/me	Ejecutar sin body.	Perfil de usuario actual simulado.
GET	/api/auth/public-key	Ejecutar sin body.	Llave publica RS256 y keyId auth-service-rsa.
GET	/api/auth/users	Ejecutar sin body.	Lista de usuarios obtenida desde ms-user.

10.2 User Service

Metodo	Endpoint	Prueba Swagger	Resultado esperado
GET	/api/users/health	Ejecutar sin body.	{"status":"UP","service":"user-service"}
POST	/api/users	Body con username, email, password, firstName, lastName y role.	Respuesta 201 con usuario creado.
GET	/api/users	Ejecutar sin body.	Lista de usuarios persistidos.
POST	/api/users/authenticate	Body: {"login":"vendedor","password":"1234"}	Usuario autenticado o 401 si las credenciales fallan.
GET	/api/users/{username}	Parametro username, por ejemplo vendedor.	Usuario encontrado o 404.

10.3 KPIs Service

Metodo	Endpoint	Prueba Swagger	Resultado esperado
GET	/api/kpis/health	Ejecutar sin body.	{"status":"UP","service":"kpis-service"}
GET	/api/kpis/summary	Ejecutar sin body.	Resumen de ventas, margen, stock critico, reclamos, ticket y satisfaccion.
GET	/api/kpis/sales/monthly	Ejecutar sin body.	Lista de ventas mensuales.
GET	/api/kpis/branches/performance	Ejecutar sin body.	Desempeno por sucursal.
GET	/api/kpis/channels	Ejecutar sin body.	Distribucion por canal de venta.
GET	/api/kpis/alerts	Ejecutar sin body.	Lista de alertas.
GET	/api/kpis/{type}	Parametro type: SUMMARY, MONTHLY_SALES, BRANCH_PERFORMANCE, SALES_CHANNELS o ALERTS.	Respuesta del KPI segun estrategia.

10.4 Report Service

Metodo	Endpoint	Prueba Swagger	Resultado esperado
GET	/api/reports/health	Ejecutar sin body.	{"status":"UP","service":"report-service"}
GET	/api/reports	Ejecutar sin body.	Lista de reportes persistidos.
POST	/api/reports	Body: {"title":"Reporte mensual","description":"Resumen de ventas","reportType":"SALES","status":"PENDING"}	Respuesta 201 con reporte creado y generatedAt.
GET	/api/reports/{id}	Parametro id, por ejemplo 1.	Reporte encontrado o 404.

Nota: el contrato publico del frontend pasa por KrakenD y BFF. Swagger prueba los microservicios directos; por eso incluye endpoints internos o directos que no necesariamente estan publicados por el gateway.